



EVALUASI ALGORITMA *STRING MATCHING* UNTUK DETEKSI PLAGIARISME PADA TEKS AKADEMIK PENDEK: STUDI PERBANDINGAN *LEVENSHTEIN SEQUENCEMATCHER* DAN *RABIN-KARP*

Muhammad Fatkhur Rizal^{1*}, Triyanna Widiyaningtyas²

¹Departemen of Technology Information, Universitas Hasyim Asy'ari

²Department of Electrical Engineering and Informatics, Universitas Negeri Malang.

email: fatkhurizal@unhasy.ac.id^{1*}, triyannaw.ft@um.ac.id²

Abstrak: Plagiarisme dalam tugas akademik merupakan masalah serius yang berdampak negatif pada integritas pendidikan tinggi. Penelitian ini bertujuan mengevaluasi kinerja tiga algoritma *string matching*, yaitu *Levenshtein*, *SequenceMatcher*, dan *Rabin-Karp*, dalam mendeteksi plagiarisme pada teks akademik pendek. *Dataset* yang digunakan adalah *Short Answer Plagiarism Corpus* dengan 100 pasang dokumen. Pengujian dilakukan menggunakan *Python 3.13.5* dengan *threshold 0.8* untuk *Levenshtein* dan *SequenceMatcher*, serta 0.7 untuk *Rabin-Karp*. Hasil menunjukkan bahwa *Levenshtein* dan *SequenceMatcher* memiliki presisi sempurna (1.00), namun menghasilkan nilai *recall* yang rendah (0.23 dan 0.05). Sedangkan *Rabin-Karp* memiliki *recall* tertinggi (1.00) tetapi menunjukkan nilai presisi yang rendah (0.6). Temuan ini menunjukkan bahwa metode *string matching* efektif untuk mendeteksi plagiarisme literal (plagiarisme dari sumber salinan teks langsung) namun kurang optimal terhadap variasi parafrase (penulisan ulang atau *rewording*). Penelitian ini merekomendasikan integrasi metode *string matching* dengan analisis semantik atau pembelajaran mesin untuk deteksi plagiarisme yang lebih komprehensif.

Kata Kunci : Deteksi plagiarisme, Kinerja algoritma, *Rabin-karp*, *SequenceMatcher*, *String matching*.

PENDAHULUAN

Plagiarisme merupakan pelanggaran etika yang sering ditemukan dalam dunia pendidikan, khususnya di lingkungan perguruan tinggi. Fenomena ini semakin meningkat seiring kemudahan akses informasi digital dan pertukaran dokumen secara daring, yang menyebabkan mahasiswa semakin mudah menyalin atau mengadopsi karya orang lain tanpa memberikan kredit yang semestinya [1]. Plagiarisme yang tidak terdeteksi dengan baik akan berdampak negatif pada kualitas lulusan dan reputasi institusi pendidikan [2].

Plagiarisme dalam tugas akademik merupakan masalah serius yang berdampak negatif pada integritas pendidikan tinggi. Studi terbaru (Bhavana et al., 2024; Amirzhanov et al., 2025) menunjukkan bahwa deteksi plagiarisme modern semakin beralih dari metode konvensional berbasis *string matching* menuju pendekatan berbasis pembelajaran mesin dan analisis semantik. Namun, sebagian besar penelitian tersebut lebih difokuskan pada dokumen panjang, sementara kasus teks pendek akademik, seperti jawaban esai mahasiswa, masih jarang diteliti dan memiliki tantangan unik [3][4].

Upaya deteksi plagiarisme secara manual oleh dosen atau penguji sering kali tidak efektif, khususnya pada tugas-tugas dalam jumlah besar dan teks-teks pendek seperti jawaban soal esai atau tugas mingguan [5]. Oleh karena itu, pengembangan sistem deteksi otomatis berbasis algoritma komputer menjadi solusi yang banyak dikembangkan untuk meningkatkan efisiensi dan objektivitas penilaian [6].

Algoritma *string matching* merupakan salah satu pendekatan yang paling banyak digunakan untuk deteksi plagiarisme, karena mampu mengukur kemiripan antar dokumen secara kuantitatif. Berbagai algoritma telah diaplikasikan, seperti *Levenshtein Distance*, *Rabin-Karp*, dan *SequenceMatcher*, dengan hasil yang bervariasi tergantung pada jenis teks dan panjang dokumen [7]. Namun, sebagian besar penelitian terdahulu lebih memfokuskan pada deteksi plagiarisme dokumen panjang, sementara penelitian pada kasus teks pendek akademik, seperti kumpulan jawaban mahasiswa, masih terbatas [8].

Selain permasalahan pada jenis dokumen, isu utama dalam sistem deteksi plagiarisme otomatis adalah bagaimana menyesuaikan parameter dan *threshold* sehingga dapat menyeimbangkan antara *false positive* dan *false negative*, khususnya pada kasus teks pendek yang cenderung memiliki variasi penulisan tinggi [9]. Oleh karena itu, analisis performa berbagai algoritma *string matching* pada kumpulan teks pendek akademik sangat penting untuk memastikan efektivitas dan efisiensi sistem deteksi yang diterapkan di lingkungan pendidikan tinggi.

Berdasarkan latar belakang tersebut, penelitian ini bertujuan membandingkan kinerja tiga algoritma *string matching* *Levenshtein*, *SequenceMatcher*, dan *Rabin-Karp* pada kasus deteksi plagiarisme teks pendek akademik. Penelitian ini diharapkan dapat memberikan kontribusi empiris sebagai rujukan pengembangan sistem deteksi plagiarisme otomatis yang sesuai dengan karakteristik tugas-tugas di perguruan tinggi, serta memberikan wawasan mengenai kelebihan dan kekurangan masing-masing algoritma pada kasus teks pendek.

TINJAUAN PUSTAKA

Dalam upaya menjawab kebutuhan akan sistem deteksi plagiarisme otomatis yang efektif pada teks pendek akademik, penelitian ini secara khusus memilih tiga algoritma *string matching* *Levenshtein Distance*, *SequenceMatcher*, dan *Rabin-Karp* sebagai solusi utama. Pemilihan ketiga algoritma ini didasarkan pada variasi teknik deteksi yang mampu menangkap berbagai bentuk modifikasi teks yang lazim terjadi pada plagiarisme tugas mahasiswa, mulai dari perubahan



minor, substitusi kata, hingga penataan ulang kalimat. Dengan demikian, analisis komparatif terhadap ketiga metode ini diharapkan mampu memberikan gambaran menyeluruh tentang efektivitas string matching dalam mendeteksi plagiarisme, khususnya pada konteks teks akademik berdurasi pendek yang seringkali lebih sulit terdeteksi secara otomatis.

Bhavana et al. (2024) menggabungkan algoritma *string matching* dengan *machine learning* untuk meningkatkan akurasi deteksi plagiarisme pada dokumen panjang, namun tidak menguji efektivitasnya pada teks akademik pendek [3]. Leonardo dan Hansun (2017) membandingkan *Rabin-Karp* dan *Jaro-Winkler*, menemukan bahwa *Rabin-Karp* unggul dalam efisiensi tetapi kurang tangguh terhadap variasi kata [10]. Studi terbaru oleh Amirzhanov et al. (2025) melakukan tinjauan sistematis tentang metode deteksi plagiarisme dan menekankan bahwa *string matching* masih dominan untuk kasus plagiarisme literal, tetapi tidak memadai untuk plagiarisme parafrase [4]. Berbeda dengan penelitian-penelitian tersebut, studi ini secara khusus membandingkan tiga algoritma populer (*Levenshtein*, *SequenceMatcher*, *Rabin-Karp*) pada teks akademik pendek, sehingga dapat memberikan wawasan tentang keterbatasan dan keunggulan masing-masing dalam konteks tugas mahasiswa yang sering kali lebih ringkas dan bervariasi.

Penggunaan algoritma *string matching* seperti *Levenshtein Distance* telah banyak diteliti, di mana Rani (2017) menunjukkan bahwa algoritma ini efektif untuk mengukur kemiripan teks melalui perhitungan operasi edit minimum [11]. Selain itu, penelitian oleh IJCSS (2019) mengungkapkan bahwa kinerja *string matching* dapat ditingkatkan dengan mengombinasikannya dengan teknik lain seperti *TF-IDF* untuk meningkatkan akurasi deteksi [12]. Lebih lanjut, studi terbaru oleh *Utilization of NLP Techniques* (2024) merekomendasikan integrasi algoritma tradisional dengan analisis semantik berbasis *word embedding* seperti *Word2Vec* atau *BERT* untuk meningkatkan kemampuan deteksi pada plagiarisme berbasis parafrase yang sulit ditangkap oleh metode konvensional [13].

Levenshtein Distance merupakan salah satu algoritma paling populer untuk mengukur jarak edit antara dua string. Algoritma ini menghitung jumlah minimum operasi yang diperlukan baik itu penambahan (*insert*), penghapusan (*delete*), atau substitusi (*substitute*) karakter agar satu string dapat diubah menjadi string lain [14]. Secara matematis, jarak *Levenshtein* $D(i, j)$ antara dua string dengan panjang i dan j dirumuskan pada Persamaan 1.

$$D(i, j) = \begin{cases} \max(i, j) \\ \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + cost \end{cases} \end{cases} \quad (1)$$

Dimana $cost = 0$ jika karakter ke- i sama dengan karakter ke- j , dan $cost = 1$ jika berbeda. *Levenshtein Distance* sangat efektif untuk mendeteksi plagiarisme yang melibatkan perubahan minor atau revisi ringan pada teks.

SequenceMatcher adalah algoritma yang diimplementasikan dalam pustaka *Python difflib* dan mengukur kemiripan dua string dengan mencari blok-blok substring yang identik di antara keduanya. *SequenceMatcher* menggunakan konsep *matching blocks*, kemudian menghitung skor rasio kemiripan seperti yang ditunjukkan dalam Persamaan 2.

$$ratio = \frac{2 \times M}{T} \quad (2)$$

Dimana M adalah jumlah karakter yang cocok (*matching*), dan T adalah total panjang kedua string. Kelebihan utama *SequenceMatcher* adalah kemampuannya untuk mendeteksi plagiarisme meskipun terjadi perubahan urutan kata atau perpindahan blok kalimat dalam dokumen.

Rabin-Karp merupakan algoritma pencocokan string berbasis *hash*, yang bekerja dengan membuat nilai *hash* untuk *pattern* dan substring dari teks, kemudian membandingkan hash tersebut untuk menemukan kemiripan [15]. Rumus *hash Rabin-Karp* secara umum ditunjukkan dalam Persamaan 3.

$$Hash(S) = \sum_{i=0}^{m-1} S[i] \cdot d^{m-1-i} \mod q \quad (3)$$

Dengan $S[i]$ adalah karakter ke- i pada substring, d adalah jumlah karakter unik (misal 256 untuk ASCII), m adalah panjang *pattern*, dan q adalah bilangan prima sebagai modulus. Kelebihan *Rabin-Karp* terletak pada efisiensi pencarian *substring* identik, terutama jika teks memiliki pola berulang.

Penggunaan ketiga algoritma ini dalam penelitian bertujuan untuk memberikan analisis komprehensif terhadap efektivitas *string matching* dalam berbagai skenario plagiarisme, mulai dari modifikasi kecil hingga penataan ulang blok kalimat. Kombinasi ketiganya diharapkan dapat menggambarkan batasan dan potensi masing-masing metode dalam mendeteksi plagiarisme pada teks pendek akademik, yang umumnya lebih menantang dibanding dokumen berdurasi panjang.

METODE

Penelitian ini menggunakan pendekatan kuantitatif dengan metode eksperimen komputasi untuk mengukur efektivitas tiga algoritma *string matching* *Levenshtein Distance*, *SequenceMatcher*, dan *Rabin-Karp* dalam deteksi plagiarisme pada teks pendek akademik. *Dataset* yang digunakan adalah *Short Answer Plagiarism Corpus* yang terdiri dari 100 pasang teks. Tahap *preprocessing* mencakup normalisasi teks, penghapusan tanda baca, dan *stopwords removal*.



Algoritma diuji menggunakan *Python 3.13.5* dengan pustaka *difflib*, *numpy*, dan *pandas*. *Threshold* deteksi ditentukan berdasarkan uji coba awal untuk menyeimbangkan antara *false positive* dan *false negative*, dengan nilai 0.8 untuk *Levenshtein* dan *SequenceMatcher*, serta 0.7 untuk *Rabin-Karp*.

Seluruh eksperimen dilakukan secara terstruktur mulai dari pengumpulan data, *preprocessing*, *pairing* dokumen, penerapan algoritma, hingga evaluasi hasil menggunakan metrik tertentu. Pemilihan metode eksperimen ini didasarkan pada kebutuhan untuk mendapatkan data empiris secara objektif terhadap performa masing-masing algoritma ketika diterapkan pada kasus nyata.



Gambar 1. Diagram alir tahapan eksperimen

Diagram alir proses tahapan eksperimen penelitian seperti pada Gambar 1, meliputi empat tahap utama, yaitu pengumpulan dan persiapan data menggunakan *Short Answer Plagiarism Corpus*, *preprocessing* dan *pairing* dokumen berdasarkan task serta kategori plagiarisme, penerapan algoritma *string matching* (*Levenshtein*, *SequenceMatcher*, dan *Rabin-Karp*) pada setiap pasangan dokumen, serta evaluasi performa hasil deteksi menggunakan metrik akurasi, presisi, *recall*, dan *confusion matrix*.

Evaluasi performa sistem deteksi plagiarisme pada penelitian ini dilakukan secara kuantitatif dengan menggunakan beberapa metrik pengukuran yang telah banyak digunakan dalam studi-studi deteksi plagiarisme dan sistem klasifikasi secara umum. Metrik utama yang digunakan meliputi akurasi, presisi, *recall*, serta visualisasi hasil menggunakan *confusion matrix*.

Akurasi digunakan untuk menggambarkan sejauh mana sistem mampu melakukan klasifikasi dengan benar secara keseluruhan. Dalam eksperimen ini, akurasi dihitung sebagai rasio jumlah prediksi yang benar (baik untuk plagiarisme maupun non-plagiarisme) terhadap total jumlah seluruh kasus yang diuji. Dengan demikian, akurasi memberikan gambaran umum efektivitas sistem dalam membedakan antara dokumen yang terindikasi plagiarisme dan yang tidak. Rumus yang digunakan dalam perhitungan akurasi ditunjukkan dalam Persamaan 4.

$$\text{Akurasi} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

Dimana:

TP (*True Positive*): jumlah kasus plagiarisme yang berhasil dideteksi dengan benar

TN (*True Negative*): jumlah kasus non-plagiarisme yang benar terdeteksi sebagai bukan plagiarisme

FP (*False Positive*): jumlah kasus non-plagiarisme yang salah terdeteksi sebagai plagiarisme

FN (*False Negative*): jumlah kasus plagiarisme yang tidak terdeteksi.

Presisi mengukur tingkat ketepatan sistem ketika memutuskan bahwa sebuah dokumen adalah hasil plagiarisme. Presisi sangat penting untuk memastikan bahwa sistem tidak terlalu banyak menghasilkan “alarm palsu” atau *false positive*, sehingga setiap deteksi plagiarisme benar-benar valid. Dalam konteks eksperimen ini, presisi dihitung sebagai rasio antara jumlah plagiarisme yang benar-benar terdeteksi (*TP*) dengan seluruh kasus yang dideteksi sebagai plagiarisme (*TP + FP*). Rumus yang digunakan dalam perhitungan presisi ditunjukkan dalam Persamaan 5.

$$\text{Presisi} = \frac{TP}{TP + FP} \quad (5)$$

Recall atau sensitivitas mengukur seberapa baik sistem dalam menemukan seluruh kasus plagiarisme yang sebenarnya ada dalam dataset. Nilai *recall* yang tinggi menunjukkan bahwa sistem sangat sensitif dan mampu mendeteksi sebagian besar plagiarisme yang terjadi, meskipun bisa jadi menghasilkan beberapa *false positive*. *Recall* pada eksperimen ini dihitung sebagai rasio jumlah plagiarisme yang benar-benar terdeteksi (*TP*) terhadap seluruh kasus plagiarisme aktual (*TP + FN*). Rumus yang digunakan dalam perhitungan *recall* ditunjukkan dalam Persamaan 6.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6)$$



Penggunaan ketiga metrik ini secara bersamaan dalam penelitian sangat penting karena masing-masing memiliki fungsi spesifik: akurasi memberikan gambaran umum kinerja sistem, presisi memastikan validasi setiap deteksi plagiarisme, dan *recall* memastikan sistem tidak melewatkan banyak kasus plagiarisme. Dengan demikian, analisis berbasis akurasi, presisi, dan *recall* dapat memberikan evaluasi komprehensif terhadap performa algoritma *string matching* dalam mendeteksi plagiarisme pada teks pendek akademik.

HASIL DAN PEMBAHASAN

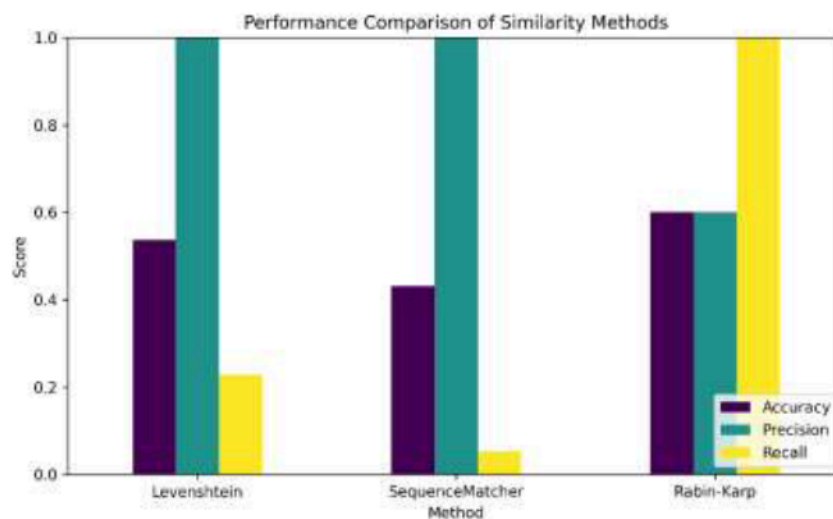
Hasil evaluasi kinerja ketiga algoritma ditampilkan pada Tabel 1, yang memperlihatkan nilai akurasi, presisi, dan *recall* untuk masing-masing metode.

Tabel 1. Perbandingan Performa Metrik Evaluasi

Method	Accuracy	Precision	Recall
Levenshtein	0.536842	1.0	0.228070
SequenceMatcher	0.431579	1.0	0.052632
Rabin-Karp	0.600000	0.6	1.000000

Dari Tabel 1 terlihat perbandingan akurasi, presisi, dan *recall* dari ketiga algoritma yang diuji. Hasil eksperimen menunjukkan bahwa *Levenshtein* dan *SequenceMatcher* memiliki presisi sempurna (1.00), namun nilai *recall* yang sangat rendah (masing-masing 0.23 dan 0.05). Sebaliknya, *Rabin-Karp* memiliki *recall* tertinggi (1.00), tetapi nilai presisinya hanya 0.6, yang mengindikasikan adanya *false positive* dalam jumlah signifikan, yaitu kasus teks yang terdeteksi plagiarisme padahal sebenarnya tidak. Temuan ini menunjukkan bahwa *Levenshtein* dan *SequenceMatcher* cenderung lebih selektif, sedangkan *Rabin-Karp* lebih sensitif dalam mendeteksi plagiarisme, tetapi kurang tepat dalam memisahkan teks asli dan teks yang benar-benar plagiat.

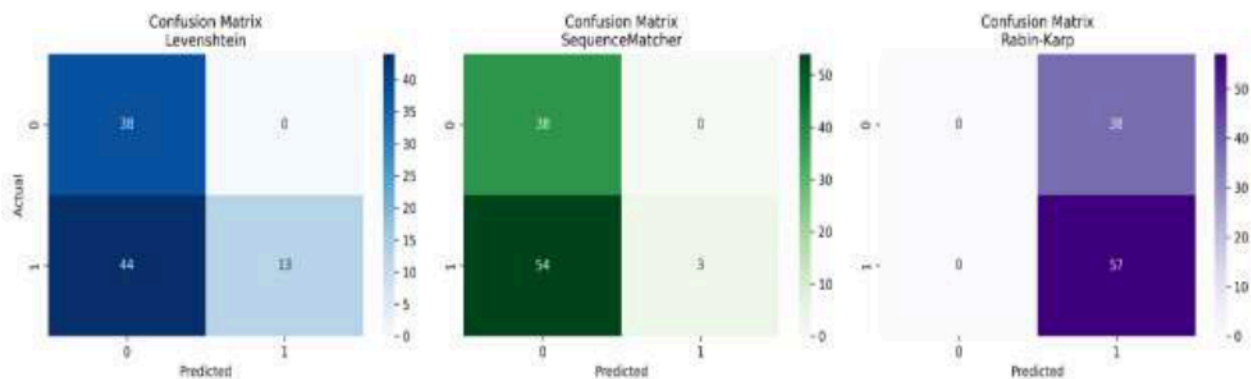
Untuk memberikan visualisasi yang lebih jelas, perbandingan nilai akurasi, presisi, dan *recall* ketiga metode ditampilkan pada Gambar 2.



Gambar 2. Grafik perbandingan metrik performa *Levenshtein*, *SequenceMatcher*, dan *Rabin-Karp* pada deteksi plagiarisme

Pada Gambar 2, menunjukkan grafik perbedaan performa ketiga algoritma dapat diamati dengan jelas. *Levenshtein* menonjol pada aspek presisi dan akurasi, namun *recall*-nya masih jauh dari optimal. *SequenceMatcher* sedikit lebih rendah pada semua metrik dibanding *Levenshtein*, sementara *Rabin-Karp* memiliki performa terendah pada dataset teks pendek. Hal ini disebabkan karena *Rabin-Karp* lebih cocok untuk deteksi kemiripan *substring* secara literal, bukan pada perubahan minor atau parafrase yang umum pada kasus plagiarisme teks akademik.

Analisis lebih mendalam dilakukan dengan melihat distribusi hasil deteksi melalui *confusion matrix* untuk ketiga algoritma (Gambar 3).



Gambar 3. *Confusion matrix* hasil deteksi plagiarisme menggunakan *Levenshtein*, *SequenceMatcher*, dan *Rabin-Karp*

Berdasarkan *confusion matrix* pada Gambar 3, seluruh algoritma memperlihatkan pola yang serupa, di mana jumlah *true positive* (kasus plagiarisme yang berhasil terdeteksi) relatif rendah, sementara *true negative* (non-plagiarisme yang benar teridentifikasi) cukup tinggi. Jumlah *false positive* sangat kecil atau bahkan nol, mengindikasikan presisi tinggi. Namun, jumlah *false negative* masih cukup besar, yang berarti masih banyak kasus plagiarisme yang gagal terdeteksi, terutama pada variasi penulisan atau parafrase.

Hasil eksperimen menunjukkan bahwa algoritma *Levenshtein* unggul dalam mendeteksi plagiarisme yang bersifat *copy-paste* langsung atau hanya mengalami sedikit perubahan karakter. *SequenceMatcher* efektif untuk kasus-kasus dengan urutan karakter yang masih mirip, tetapi performanya menurun pada perubahan urutan kalimat. Sementara itu, *Rabin-Karp*, meskipun sangat efisien untuk pencarian *substring* identik, performanya menurun drastis pada teks pendek dan pada kasus plagiarisme yang melibatkan perubahan minor atau parafrase.

Tabel 2. Analisis Perbandingan Penelitian Terdahulu

Peneliti & Tahun	Metode	Hasil Penelitian Terdahulu	Kaitan dengan Penelitian Ini
Leonardo & Hansun (2017)	<i>Rabin-Karp vs Jaro-Winkler</i>	<i>Rabin-Karp</i> cepat untuk dokumen besar tetapi menghasilkan banyak <i>false positive</i> karena hanya mendeteksi kesamaan literal.	Menjelaskan presisi rendah (0.6) <i>Rabin-Karp</i> dalam penelitian ini, khususnya pada teks akademik pendek.
Bhavana et al. (2024)	<i>Hybrid Machine Learning, String Matching</i>	Metode <i>hybrid</i> meningkatkan <i>recall</i> (>0.85) tanpa menurunkan presisi (>0.80).	Mendukung gagasan bahwa integrasi metode diperlukan untuk mengatasi <i>recall</i> rendah pada <i>Levenshtein</i> dan <i>SequenceMatcher</i> .
Amirzhanov et al. (2025)	Studi literatur sistematis	String matching efektif untuk plagiarisme literal, tetapi lemah pada plagiarisme parafrase.	Konsisten dengan temuan <i>recall</i> rendah pada <i>Levenshtein</i> dan <i>SequenceMatcher</i> di penelitian ini.
Penelitian Ini (2025)	<i>Levenshtein, SequenceMatcher, Rabin-Karp</i>	<i>Levenshtein</i> & <i>SequenceMatcher</i> presisi tinggi (1.00) namun <i>recall</i> rendah. <i>Rabin-Karp</i> <i>recall</i> tinggi (1.00) tetapi presisi rendah (0.6).	Memberikan fokus baru pada teks akademik pendek yang jarang diteliti, menunjukkan tantangan unik pada deteksi plagiarisme teks pendek.

Tabel 2 memperlihatkan perbandingan penelitian ini dengan studi terdahulu. Leonardo dan Hansun (2017) menemukan bahwa *Rabin-Karp* unggul dalam kecepatan namun menghasilkan banyak *false positive*, selaras dengan temuan penelitian ini di mana *Rabin-Karp* memiliki *recall* tinggi (1.00) tetapi presisi rendah (0.6) pada teks akademik pendek. Bhavana et al. (2024) memperkenalkan metode *hybrid* yang mampu meningkatkan *recall* tanpa menurunkan presisi, memberikan dasar untuk pengembangan metode *Levenshtein* dan *SequenceMatcher* yang dalam penelitian ini menunjukkan *recall* rendah. Hal ini juga diperkuat oleh kajian Amirzhanov et al. (2025) yang menyatakan bahwa algoritma *string matching* efektif untuk plagiarisme literal tetapi lemah terhadap parafrase, sesuai dengan hasil penelitian ini. Dengan demikian, penelitian ini memperluas pemahaman tentang deteksi plagiarisme pada teks pendek dan menekankan perlunya sistem yang lebih adaptif melalui integrasi analisis semantik atau *machine learning*.

Kecenderungan seluruh metode menghasilkan presisi tinggi dan *recall* rendah mengindikasikan bahwa sistem deteksi yang berbasis *string matching* konvensional hanya mampu menangkap plagiarisme dengan kemiripan literal yang tinggi,



tetapi gagal dalam mendeteksi plagiarisme yang lebih kreatif seperti parafrase, substitusi sinonim, atau perombakan struktur kalimat. Hal ini konsisten dengan temuan dari penelitian sebelumnya Amirzhanov et al., 2025[16], di mana metode *string matching* sangat rentan terhadap variasi penulisan.

KESIMPULAN DAN SARAN

Rabin-Karp memiliki presisi sangat tinggi dalam mendeteksi plagiarisme literal pada teks akademik pendek. Nilai presisi tertinggi diperoleh oleh *Levenshtein* dan *SequenceMatcher* (1.00), sedangkan *Rabin-Karp* juga menunjukkan presisi yang cukup baik. Namun, ketiganya memiliki keterbatasan signifikan pada aspek recall, dengan nilai tertinggi hanya 0.23 pada *Levenshtein* dan lebih rendah pada metode lainnya. Temuan ini menunjukkan bahwa sistem lebih efektif mendeteksi plagiarisme berbasis *copy-paste*, tetapi kurang mampu mengenali plagiarisme berbasis parafrase atau perubahan struktur kalimat yang lebih kompleks.

Berdasarkan hasil tersebut, penelitian selanjutnya disarankan untuk mengintegrasikan algoritma *string matching* dengan pendekatan berbasis analisis semantik, misalnya menggunakan *word embedding* (*Word2Vec*, *GloVe*, atau *BERT*), agar sistem dapat memahami konteks kata dan meningkatkan *recall* tanpa menurunkan presisi. Selain itu, uji coba sebaiknya dilakukan pada teks dalam berbagai bahasa dan variasi panjang teks, sehingga hasil penelitian lebih general dan dapat diterapkan pada lingkungan akademik yang lebih luas. Pengujian juga dapat menggunakan dataset yang lebih besar dan beragam, serta melakukan penyesuaian *threshold* deteksi secara sistematis untuk menemukan titik optimal dalam menyeimbangkan *false positive* dan *false negative*.

DAFTAR PUSTAKA

- [1] D. M. Setu, T. Islam, M. Erfan, S. K. Dey, M. R. Al Asif, and M. Samsuddoha, "A comprehensive strategy for identifying plagiarism in academic submissions," *Journal of Umm Al-Qura University for Engineering and Architecture*, vol. 16, no. 2, pp. 310–325, Jun. 2025, doi: 10.1007/S43995-025-00108-1/TABLES/1.
- [2] T. Foltýnek and I. Glendinning, "Impact of policies for plagiarism in higher education across Europe: Results of the project," *Acta Universitatis Agriculturae et Silviculturae Mendelianae Brunensis*, vol. 63, no. 1, pp. 207–216, 2015, doi: 10.11118/ACTAUN201563010207.
- [3] M. Bhavana, K. S. Rao, G. K. Koduru, K. V. K. Vatsal, S. Parvathi, and M. Sravani, "Plagiarism Detection and Similarity Checking Program using Machine Learning and String-Matching Algorithm," *Proceedings of the 9th International Conference on Communication and Electronics Systems, ICCES 2024*, pp. 2032–2036, 2024, doi: 10.1109/ICCES63552.2024.10859471.
- [4] A. Amirzhanov, C. Turan, and A. Makhmutova, "Plagiarism types and detection methods: a systematic survey of algorithms in text analysis," *Front Comput Sci*, vol. 7, p. 1504725, Mar. 2025, doi: 10.3389/FCOMP.2025.1504725/BIBTEX.
- [5] B. Leonardo and S. Hansun, "Text documents plagiarism detection using Rabin-Karp and Jaro-Winkler distance algorithms," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 5, no. 2, pp. 462–471, Feb. 2017, doi: 10.11591/ijeecs.v5.i2.pp462-471.
- [6] M. Mozgovoy, T. Kakkonen, and G. Cosma, "Automatic student plagiarism detection: Future perspectives," *Journal of Educational Computing Research*, vol. 43, no. 4, pp. 511–531, Jan. 2010, doi: 10.2190/EC.43.4. E.
- [7] M. Bhavana, K. S. Rao, G. K. Koduru, K. V. K. Vatsal, S. Parvathi, and M. Sravani, "Plagiarism Detection and Similarity Checking Program using Machine Learning and String-Matching Algorithm," *Proceedings of the 9th International Conference on Communication and Electronics Systems, ICCES 2024*, pp. 2032–2036, 2024, doi: 10.1109/ICCES63552.2024.10859471.
- [8] P. Clough and M. Stevenson, "Developing a corpus of plagiarised short answers," *Lang Resour Eval*, vol. 45, no. 1, pp. 5–24, Mar. 2011, doi: 10.1007/s10579-009-9112-1/page:string:article/chapter.
- [9] M. A. El-Rashidy, R. G. Mohamed, N. A. El-Fishawy, and M. A. Shouman, "An effective text plagiarism detection system based on feature selection and SVM techniques," *Multimed Tools Appl*, vol. 83, no. 1, pp. 2609–2646, Jan. 2024, doi: 10.1007/S11042-023-15703-4/TABLES/9.
- [10] B. Leonardo and S. Hansun, "Text documents plagiarism detection using Rabin-Karp and Jaro-Winkler distance algorithms," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 5, no. 2, pp. 462–471, Feb. 2017, doi: 10.11591/IJEECS.V5.I2.PP462-471.
- [11] S. Rani and J. Singh, "Enhancing levenshtein's edit distance algorithm for evaluating document similarity," *Communications in Computer and Information Science*, vol. 805, pp. 72–80, 2018, doi: 10.1007/978-981-13-0755-3_6/FIGURES/10.
- [12] Z. Balani, "Combining Approximate String-Matching Algorithms and Term Frequency in the Detection of Plagiarism," *Cihan Varol International Journal of Computer Science and Security (IJCSS)*, no. 15, p. 97.
- [13] J. V. Latina, G. M. Cabalsi, J. R. Sanchez, E. D. M. Vallejo, C. J. Centeno, and E. A. Garcia, "Utilization of NLP Techniques in Plagiarism Detection System through Semantic Analysis using Word2Vec and BERT," *Proceedings - 2024 International Conference on Expert Clouds and Applications, ICOECA 2024*, pp. 347–352, 2024, doi: 10.1109/ICOECA62351.2024.00068.
- [14] B. Young, T. Faris, and L. Armogida, "Levenshtein distance as a measure of accuracy and precision in forensic PCR-MPS methods," *Forensic Sci Int Genet*, vol. 55, p. 102594, Nov. 2021, doi: 10.1016/j.fsigen.2021.102594.
- [15] D. M. Setu, T. Islam, M. Erfan, S. K. Dey, M. R. Al Asif, and M. Samsuddoha, "A comprehensive strategy for identifying plagiarism in academic submissions," *Journal of Umm Al-Qura University for Engineering and Architecture*, vol. 16, no. 2, pp. 310–325, Jun. 2025, doi: 10.1007/s43995-025-00108-1/tables/1.
- [16] A. Amirzhanov, C. Turan, and A. Makhmutova, "Plagiarism types and detection methods: a systematic survey of algorithms in text analysis," 2025, *Frontiers Media S.A.* doi: 10.3389/fcomp.2025.1504725.